

HTTP Log Analysis Using Splunk SIEM

SOC Analyst Project Report

Author: SOC Analyst | Date: May 2026
Tool: Splunk Enterprise | Log Type: HTTP (http.log.gz)

Objective	Analyse HTTP logs to detect web threats, anomalies, and attack patterns
Platform	Splunk Enterprise — Search & Reporting
Source File	http.log.gz
Source Type	http logs
Total Events	2,043,366 events across the full analysis window
Key Findings	XSS, SQL Injection, Directory Traversal, Nessus scanning, Admin brute-force

SECTION 1

Introduction

HTTP (Hypertext Transfer Protocol) log files contain rich information about web server activity — requests, responses, URIs, user agents, status codes, session identifiers, and more. As a SOC analyst, I treat HTTP logs as a primary telemetry source for detecting web-layer attacks and insider threats.

In this project I uploaded a sample HTTP log dataset into **Splunk Enterprise** and performed a multi-stage investigation covering: field extraction, top-URI analysis, traffic pattern anomalies, session-level profiling, and manual threat hunting through the raw event data. The goal was to surface real attack indicators and document my analyst workflow end-to-end.

SECTION 2

Step 1 & 2 — Uploading Logs & Analysing Top URLs

2.1 Uploading the HTTP Log File

I started by adding the sample HTTP log file (**http.log.gz**) to Splunk, assigning it the source type **http logs**. Splunk indexed all 2,043,366 events and made them immediately searchable. In the field extractor (IFX) I highlighted four key fields from a sample event to drive the rest of my investigation:

- **src_ip** — Source IP of the HTTP client
- **src_port** — Client-side ephemeral port
- **dst_ip** — Server / destination IP
- **dst_port** — Destination port (typically 80 or 443)

2.2 Top URLs and Endpoints Accessed

With fields in place I ran a **top uri** query to find the most frequently requested endpoints. The results immediately highlighted **/servlet/admin** as the highest-hit URI (98 hits, 67.18%), alongside multiple URL-encoded paths suggesting automated tool activity rather than legitimate browser traffic.

```
source="http.log.gz" sourcetype="http logs" | top limit=10 uri
```


SECTION 3

Step 3 — Unusual Patterns in File Transfer & Traffic Distribution

To uncover temporal anomalies I ran a timechart breaking down URI access by hour. This produces a matrix of endpoint activity over time, making traffic spikes immediately visible.

```
source="http.log.gz" sourcetype="http logs" | timechart span=1h count by uri
```

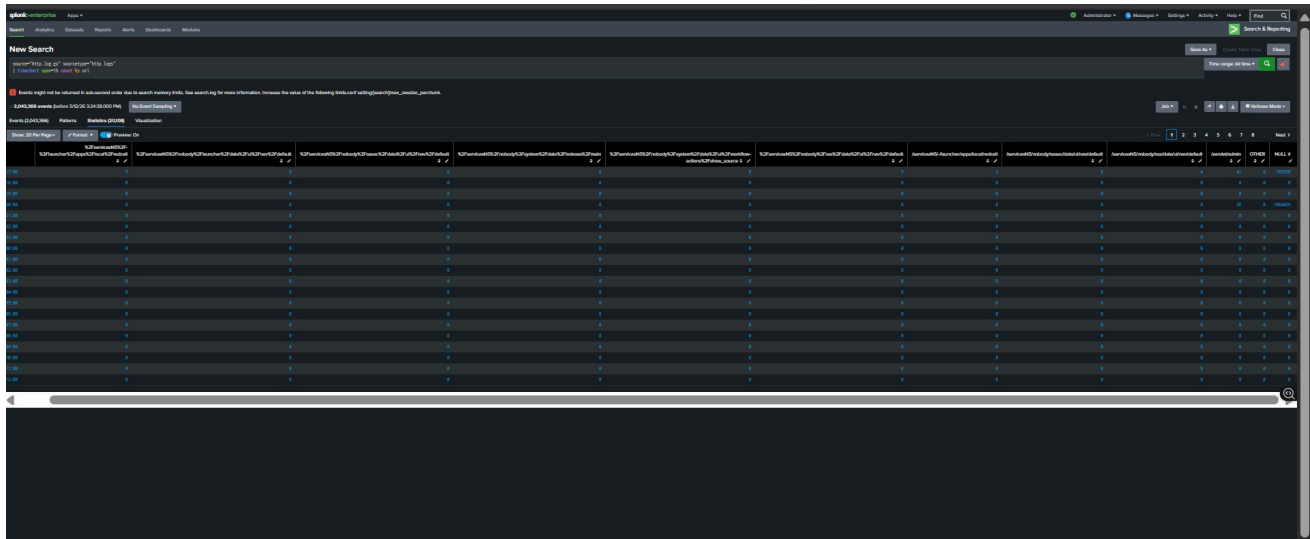


Figure 2 — Hourly web traffic distribution per URI. Each column is an endpoint; values represent request frequency per hour. High counts or unusual endpoints indicate scanning, brute-force, or abnormal behaviour.

What stood out:

OTHER (very high)

Mass scanning or fuzzing — a catch-all bucket catching high-volume hits across undefined/obfuscated endpoints. This volume pattern is characteristic of automated web scanners.

/servlet/admin activity

Repeated admin-endpoint hits distributed across multiple hours indicate a sustained brute-force or credential-stuffing campaign rather than a single burst.

The combination of a dominant OTHER column and repeated /servlet/admin probing across multiple hours is a strong indicator of **automated web application scanning** — not legitimate user behaviour.

Step 4 — Session Duration Analysis & Threat Discovery

4.1 Average Session Duration per User

I used **eventstats** to compute session duration per session ID and then averaged across users. This reveals whether sessions are unusually short (scripted/automated) or suspiciously long (persistent attacker presence).

```
source="http.log.gz" sourcetype="http logs" | eventstats range(_time) as session_duration by session_id | stats avg(session_duration) as avg_session_duration by user
```

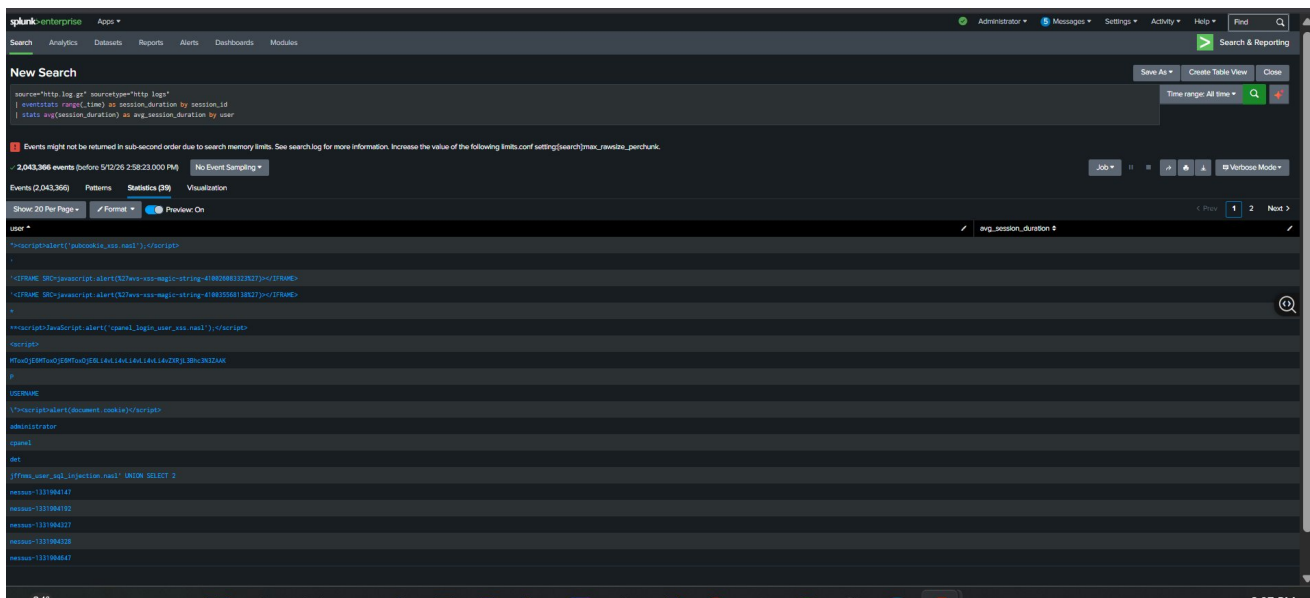


Figure 3 — Session duration and access pattern analysis output, revealing multiple attack payloads embedded in the raw HTTP event data.

4.2 Threats Uncovered in the Raw Event Data

Digging into the raw events behind this output, I identified four distinct attack categories present in the logs:

XSS Attacks

Payloads such as `<script>alert('pubcookie_xss.nasl');</script>` and `<IFRAME SRC=javascript:alert(...)></IFRAME>` are classic Cross-Site Scripting injection attempts targeting vulnerable input fields.

SQL Injection

The payload `jffnms_user_sql_injection.nasl' UNION SELECT 2` is a textbook UNION-based SQL injection attempt designed to extract data from backend databases.

Directory Traversal

A Base64-encoded payload decoded to `../../../../etc/passwd` — a path traversal attack attempting to read the system password file.

Nessus Scanning

Session IDs such as `nessus-1331904147` and `nessus-1331904192` identify automated probing by the Nessus vulnerability scanner — not a human attacker but a tool systematically testing all known CVEs.

Overall SOC-Level Conclusion

Across all four analysis steps, the HTTP logs painted a clear picture: this is **active web application probing and scanning**, not normal user traffic. The activity profile is consistent with an automated security or attack tool — most likely Nessus or a similar framework — running a comprehensive scan against the web server.

Finding	Indicator	Severity
XSS Attempts	Script/IFRAME payloads in request parameters	HIGH
SQL Injection	UNION SELECT payloads in URI	HIGH
Directory Traversal	Encoded ../../etc/passwd paths	HIGH
Admin Brute-Force	/servlet/admin — 98 hits, top URI by far	HIGH
Nessus Scanning	nessus-* session IDs across event stream	MEDIUM
Mass Fuzzing	OTHER bucket — abnormally high volume	MEDIUM

From a SOC perspective the recommended actions are:

- Immediately block the source IPs responsible for the admin endpoint probing.
- Investigate whether any XSS or SQL injection payloads succeeded — review HTTP response codes alongside the requests.
- Correlate the Nessus session IDs with authorised vulnerability scan schedules; if no scan was scheduled, treat this as a malicious external probe.
- Add a WAF rule to block directory traversal patterns and enforce input validation on all web application endpoints.
- Baseline normal session duration and alert on outliers — sessions lasting seconds with high request counts are a strong automated-attack signal.

Analysing HTTP log files through Splunk SIEM provides invaluable visibility into web-layer threats. By combining field extraction, SPL queries, timechart analysis, and raw event inspection, I was able to identify multiple concurrent attack techniques in a single dataset — demonstrating the power of SIEM-driven investigation for proactive threat detection and incident response.